

CSC-3TC33

Algorithmique et fondements de l'informatique

Concevoir un programme - partie B

Bertrand Meyer

Automne 2024

Stratégies algorithmiques

Diviser les problèmes en sous tâches

Règle de base

Pour résoudre un problème :

- Identifier des sous-tâches.
- Diviser le code en petites composantes élémentaires.
 - plus facile à tester, vérifier & maintenir
 - code plus lisible et plus facile à comprendre.

Diviser les problèmes en sous tâches

Règle de base

Pour résoudre un problème :

- Identifier des sous-tâches.
- Diviser le code en petites composantes élémentaires.
 - plus facile à tester, vérifier & maintenir
 - code plus lisible et plus facile à comprendre.

Exemple

Résoudre un système d'équations par la méthode du *pivot de Gauss*

- Dans un cours de math : une grosse procédure, compliquée à suivre
- Dans un code : nombreuses sous-fonctions (par ex : opérations sur les lignes, recherche de pivot, etc)

Le retour sur trace

Principe général du retour sur trace

But
Résoudre un problème de
satisfaction de contraintes.



Principe général du retour sur trace

But

Résoudre un problème de
satisfaction de contraintes.

Méthode

Construction *incrémentale* d'une
solution réalisable.



Principe général du retour sur trace

But

Résoudre un problème de
satisfaction de contraintes.

Méthode

Construction **incrémentale** d'une
solution réalisable.

Interruption dès qu'il est certain
que la solution partielle ne peut
être complétée.

Rebroussement pour changer la
dernière décision prise.



Exemple : le sudoku

	8					3	
2			6	7			1
	6		2	1		7	
5							3
9			7	5			8
4		1	3	9	7		6
	2					1	
8		3	1		6	5	9

Décisions : 81 cases.

Choix : valeur parmi $\llbracket 1, 9 \rrbracket$.

Vérification : le même nombre n'apparaît pas deux fois par ligne, colonne et carré.

Résolution : On remplit une case après l'autre, on revient en arrière si le critère à vérifier est violé.

Code générique

```
def solve(s):
    for d in decisions:
        if s[d] == 0:
            for c in range(1,choix(d)+1):
                s[d] = c
                if check(s) and solve(s):
                    return True
            s[d]=0
        return False
    return True
```

Le tableau **s** contient une solution partielle (où 0 désigne un choix non fait).

Valeur de retour de **solve** : **True** si une solution est trouvée.

check renvoie **False** si la solution partielle ne peut être complétée.

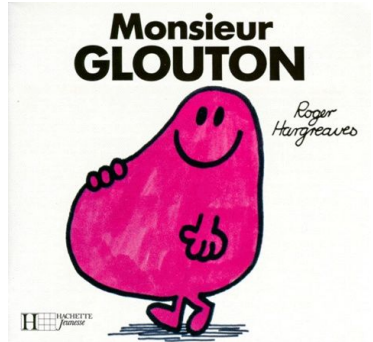
Sous-structures optimales

Définition

Un problème d'optimisation présente la propriété de **sous-structure optimale** si une solution optimale d'une instance peut se déduire des solutions optimales d'instances plus petites.

Définition

Une stratégie algorithmique **gloutonne** consiste à rechercher une solution réalisable ou un optimum global par une suite de décisions qui, à chaque étape, réalisent un optimum local.



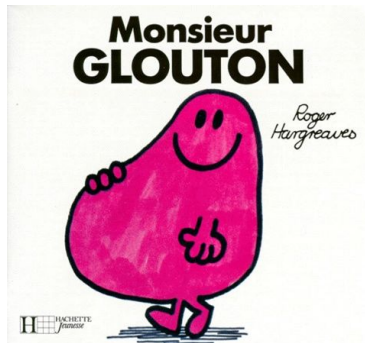
Algorithmes gloutons

Définition

Une stratégie algorithmique **gloutonne** consiste à rechercher une solution réalisable ou un optimum global par une suite de décisions qui, à chaque étape, réalisent un optimum local.

Mise en garde

Pas de garantie automatique d'optimalité! Il faut une preuve.



Le rendu du monnaie : description

Entrées : Entier $m \in \mathbb{N}$ (le montant à rendre),
tuple $(s_1, \dots, s_k) \subseteq \mathbb{N}^k$ (les pièces possibles).

Préconditions : $s_1 > s_2 > \dots > s_k > 0$

Valeur de retour : Tuple $(a_1, \dots, a_k) \in \mathbb{N}^k$

Postconditions : $m = a_1 s_1 + \dots + a_k s_k$
 $\rho(m) = \sum_{i=1}^k a_i$ est aussi petit que possible.



Rendu de monnaie glouton

```
def rendu(m, S):  
    a = [0] * len(S)  
    for i,s in enumerate(S):  
        while m >= s:  
            m -= s  
            a[i] += 1  
    if m != 0:  
        raise  
        ↳ Exception("Desolé,  
        ↳ rendu impossible  
        ↳ !")  
    return a
```

Principe

À chaque étape, on rend la pièce la plus grosse.

Rendu de monnaie glouton

```
def rendu(m, S):  
    a = [0] * len(S)  
    for i,s in enumerate(S):  
        while m >= s:  
            m -= s  
            a[i] += 1  
    if m != 0:  
        raise  
        ↳ Exception("Desolé,  
        ↳ rendu impossible  
        ↳ !")  
    return a
```

Principe

À chaque étape, on rend la pièce la plus grosse.

Contre exemple : non optimalité

Prendre : $m = 12$ et $S = (5, 4, 1)$

Rendu de monnaie glouton

```
def rendu(m, S):  
    a = [0] * len(S)  
    for i,s in enumerate(S):  
        while m >= s:  
            m -= s  
            a[i] += 1  
    if m != 0:  
        raise  
        ↳ Exception("Desolé,  
        ↳ rendu impossible  
        ↳ !")  
    return a
```

Principe

À chaque étape, on rend la pièce la plus grosse.

Contre exemple : non optimalité

Prendre : $m = 12$ et $S = (5, 4, 1)$

$$m = \underbrace{5 + 5 + 1 + 1}_{4 \text{ pièces}}$$

Rendu de monnaie glouton

```
def rendu(m, S):  
    a = [0] * len(S)  
    for i,s in enumerate(S):  
        while m >= s:  
            m -= s  
            a[i] += 1  
    if m != 0:  
        raise  
        ↳ Exception("Desolé,  
        ↳ rendu impossible  
        ↳ !")  
    return a
```

Principe

À chaque étape, on rend la pièce la plus grosse.

Contre exemple : non optimalité

Prendre : $m = 12$ et $S = (5, 4, 1)$

$$\begin{aligned} m &= \underbrace{5 + 5 + 1 + 1}_{4 \text{ pièces}} \\ &= \underbrace{4 + 4 + 4}_{\rho(12)=3} \text{ (OPT)} \end{aligned}$$

Sélection d'activité : description

Entrées : Liste de couples $[(d_i, f_i)]_{1 \leq i \leq k} \subseteq (\mathbb{N} \times \mathbb{N})^k$

Activités **programmées** entre les temps d_i et f_i .

Préconditions : $\forall i \in \llbracket 1, k \rrbracket, d_i < f_i$.

Valeur de retour : Liste de couples $[(d_i, f_i)]_{i \in I}$ où

$I = (i_1, i_2, \dots, i_\ell) \in \llbracket 1, k \rrbracket^\ell$

Activités **compatibles** entre elles aussi nombreuses que possible

Postconditions : $\forall j \in \llbracket 1, \ell - 1 \rrbracket, f_{i_j} \leq d_{i_{j+1}}$

ℓ est aussi grand que possible.



Sélection d'activité : idée

Idée

On ne perd rien à sélectionner l'activité qui se termine en premier.

Il suffit ensuite de rechercher une sélection optimale parmi les activités compatibles restantes.

Sélection d'activité : idée

Idée

On ne perd rien à sélectionner l'activité qui se termine en premier. Il suffit ensuite de rechercher une sélection optimale parmi les activités compatibles restantes.

Démonstration.

Soit $O = [a_{i_1}, a_{i_2}, \dots, a_{i_\ell}]$ une sortie optimale



Sélection d'activité : idée

Idée

On ne perd rien à sélectionner l'activité qui se termine en premier. Il suffit ensuite de rechercher une sélection optimale parmi les activités compatibles restantes.

Démonstration.

Soit $O = [a_{i_1}, a_{i_2}, \dots, a_{i_\ell}]$ une sortie optimale



Soit a_ℓ l'activité finissant en premier



Sélection d'activité : idée

Idée

On ne perd rien à sélectionner l'activité qui se termine en premier. Il suffit ensuite de rechercher une sélection optimale parmi les activités compatibles restantes.

Démonstration.

Soit $O = [a_{i_1}, a_{i_2}, \dots, a_{i_\ell}]$ une sortie optimale



Soit a_ℓ l'activité finissant en premier



Alors $O \setminus \{a_{i_1}\} \cup \{a_\ell\}$ est aussi faisable.



et sélectionne autant d'activités.



Sélection d'activité gloutonne

```
def selection(A):  
    A.sort(key=lambda a:a[1])  
    O = []  
    last_f = -sys.maxsize  
    for (d,f) in A:  
        if d >= last_f:  
            O.append((d,f))  
            last_f = f  
    return O
```

On trie les activités par ordre de temps final croissant.
On sélectionne de façon gloutonne les activités possibles.

Stratégie « diviser pour régner »

Définition

Une stratégie algorithmique **diviser pour régner** consiste à

1. décomposer une instance d'un problème en **deux sous-instances** (ou plus),
2. résoudre **récurivement** ces sous-instances,
3. **combiner** les solutions de ces sous-instances pour construire la solution de l'instance initiale.



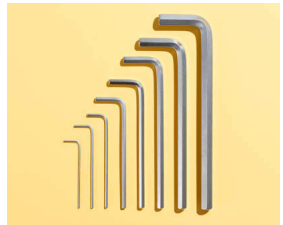
L'exponentiation rapide : description

Entrées : Élément a , entier naturel n

Préconditions : Il existe une loi de composition interne $\cdot$ sur a (produit).

Valeur de retour : Élément b de même type que a

Postconditions : $b = a^n = \underbrace{a \cdot a \cdots a}_{n \text{ fois}}$



L'exponentiation rapide : idée

Idée

$$a^n = \begin{cases} (a^{n'}) \cdot (a^{n'}) & \text{si } n = 2n' \quad (\text{Cas pair}), \\ a \cdot (a^{n'}) \cdot (a^{n'}) & \text{si } n = 2n' + 1 \quad (\text{Cas impair}). \end{cases}$$

L'exponentiation rapide : implémentation

```
def square_and_multiply(a, n):  
    if n==0:  
        return 1  
    b = square_and_multiply(a, n//2)  
    if n%2 == 0:  
        return b * b  
    else:  
        return a * b * b
```

Complexité :???

L'exponentiation rapide : implémentation

```
def square_and_multiply(a, n):  
    if n==0:  
        return 1  
    b = square_and_multiply(a, n//2)  
    if n%2 == 0:  
        return b * b  
    else:  
        return a * b * b
```

Complexité : $O(\log_2 n)$.

L'exponentiation rapide : mauvaise implémentation

```
def exp(a, n):  
    if n == 0:  
        return 1  
    if n%2 == 0:  
        return exp(a, n//2) *  
            ↪ exp(a, n//2)  
    else:  
        return a * exp(a, n//2) *  
            ↪ exp(a, n//2)
```

Complexité :???

L'exponentiation rapide : mauvaise implémentation

```
def exp(a, n):  
    if n == 0:  
        return 1  
    if n%2 == 0:  
        return exp(a, n//2) *  
            ↪ exp(a, n//2)  
    else:  
        return a * exp(a, n//2) *  
            ↪ exp(a, n//2)
```

Complexité : $O(n)$.

Mise en garde

On a vite fait de se tromper!
Appel double de `exp` : pas de gain.

Le tri : description du problème

Entrée : Tableau T de longueur n .

Préconditions : Il existe un ordre total $\leq$ sur les éléments de T .

Postconditions : $\forall i \in \llbracket 0, n - 2 \rrbracket, T[i] \leq T[i + 1]$

Le tableau T est une permutation du tableau initial.



Complexité minimal des tris comparatifs

Modèle

Un algorithme de **tri comparatif** est un algorithme dont l'exécution ne dépend que des résultats de comparaison de paires d'éléments du tableau.

Complexité minimal des tris comparatifs

Modèle

Un algorithme de **tri comparatif** est un algorithme dont l'exécution ne dépend que des résultats de comparaison de paires d'éléments du tableau.

Théorème

Tout algorithme de tri comparatif effectue $\Omega(n \log_2 n)$ opérations dans le pire des cas.

Complexité minimal des tris comparatifs

Modèle

Un algorithme de **tri comparatif** est un algorithme dont l'exécution ne dépend que des résultats de comparaison de paires d'éléments du tableau.

Théorème

Tout algorithme de tri comparatif effectue $\Omega(n \log_2 n)$ opérations dans le pire des cas.

Démonstration.

Selon l'état d'entrée, il y a $n!$ permutations de l'entrée possibles en sortie.

Complexité minimal des tris comparatifs

Modèle

Un algorithme de **tri comparatif** est un algorithme dont l'exécution ne dépend que des résultats de comparaison de paires d'éléments du tableau.

Théorème

Tout algorithme de tri comparatif effectue $\Omega(n \log_2 n)$ opérations dans le pire des cas.

Démonstration.

Selon l'état d'entrée, il y a $n!$ permutations de l'entrée possibles en sortie. Or, si d comparaisons ont été effectuées, il y a au plus 2^d états de sortie possible.

Complexité minimal des tris comparatifs

Modèle

Un algorithme de **tri comparatif** est un algorithme dont l'exécution ne dépend que des résultats de comparaison de paires d'éléments du tableau.

Théorème

Tout algorithme de tri comparatif effectue $\Omega(n \log_2 n)$ opérations dans le pire des cas.

Démonstration.

Selon l'état d'entrée, il y a $n!$ permutations de l'entrée possibles en sortie. Or, si d comparaisons ont été effectuées, il y a au plus 2^d états de sortie possible.

Ainsi tout algorithme correct satisfait $2^d \geq n!$, ce qui implique (avec la formule de Stirling)

$$d \geq \log_2(n!) = \Omega(n \log_2 n).$$

1945



John von Neumann
(1903-57)

Principe

Diviser Calculer la demi taille m du tableau T à trier

Régner Trier chaque demi-tableau $T[0 : m[$ et $T[m : n[$.

Combiner Fusionner en dépilant le plus petit des deux éléments les plus à gauche des sous-tableaux.

Tri fusion : illustration

Tableau initial :

L	C	Y	P	T	D	V	A	M	J	S
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Tri fusion : illustration

Tableau initial :

L	C	Y	P	T	D	V	A	M	J	S
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Diviser

L	C	Y	P	T
lft[0]	lft[1]	lft[2]	lft[3]	lft[4]

D	V	A	M	J	S
rght[5]	rght[6]	rght[7]	rght[8]	rght[9]	rght[10]

Tri fusion : illustration

Tableau initial :

L	C	Y	P	T	D	V	A	M	J	S
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Régner : tri récursif à gauche

C	L	P	T	Y
lft[0]	lft[1]	lft[2]	lft[3]	lft[4]

D	V	A	M	J	S
rght[5]	rght[6]	rght[7]	rght[8]	rght[9]	rght[10]

Tri fusion : illustration

Tableau initial :

L	C	Y	P	T	D	V	A	M	J	S
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

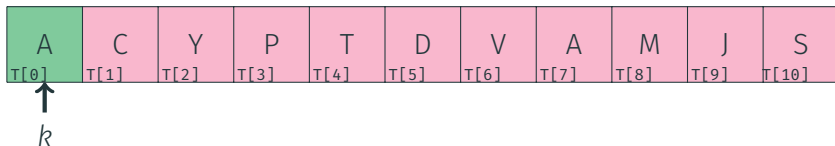
Régner : tri récursif à droite

C	L	P	T	Y
lft[0]	lft[1]	lft[2]	lft[3]	lft[4]

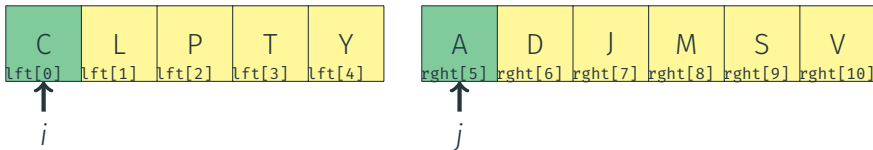
A	D	J	M	S	V
rght[5]	rght[6]	rght[7]	rght[8]	rght[9]	rght[10]

Tri fusion : illustration

Tableau initial :

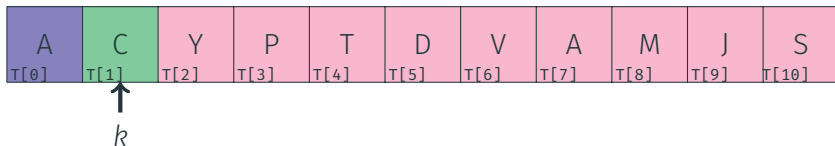


Combiner : fusion des tableaux triés

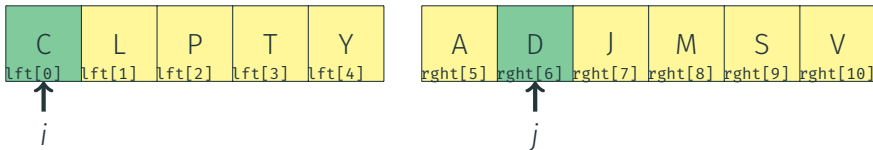


Tri fusion : illustration

Tableau initial :

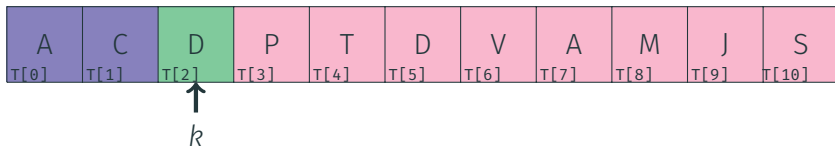


Combiner : fusion des tableaux triés

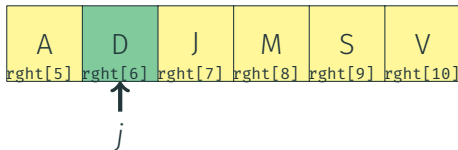
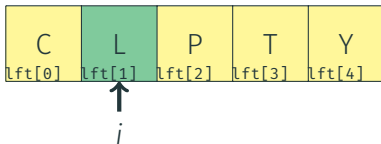


Tri fusion : illustration

Tableau initial :

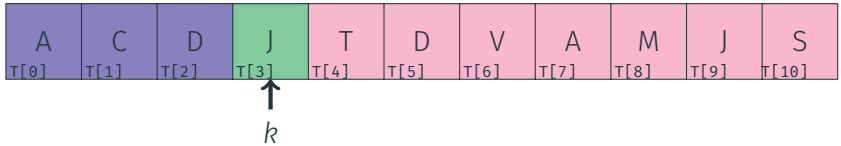


Combiner : fusion des tableaux triés

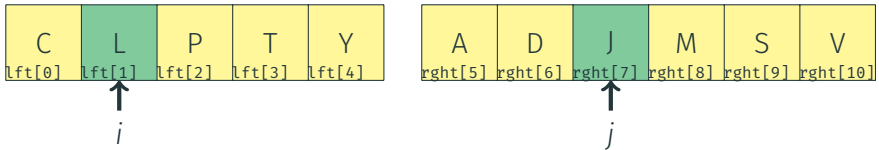


Tri fusion : illustration

Tableau initial :

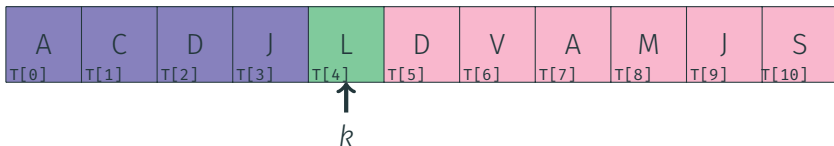


Combiner : fusion des tableaux triés

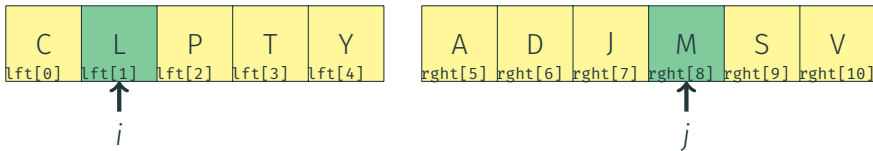


Tri fusion : illustration

Tableau initial :

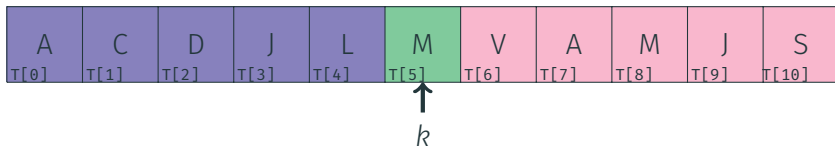


Combiner : fusion des tableaux triés

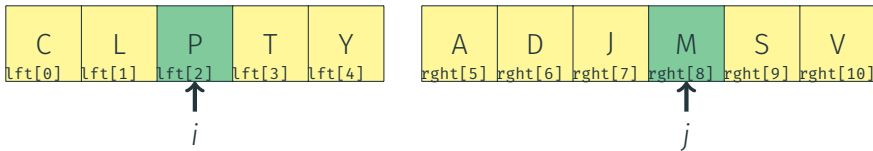


Tri fusion : illustration

Tableau initial :



Combiner : fusion des tableaux triés

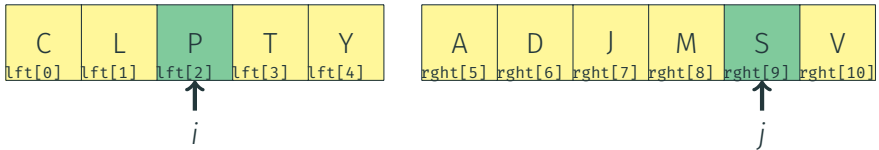


Tri fusion : illustration

Tableau initial :

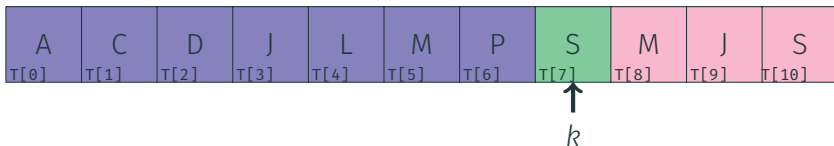


Combiner : fusion des tableaux triés

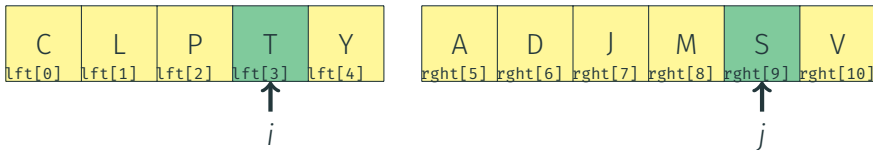


Tri fusion : illustration

Tableau initial :

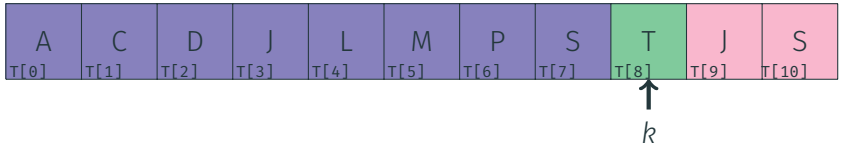


Combiner : fusion des tableaux triés

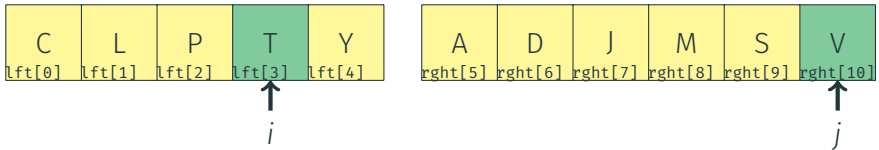


Tri fusion : illustration

Tableau initial :

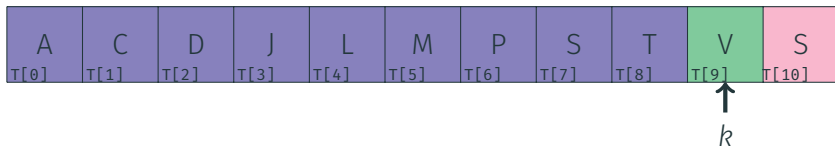


Combiner : fusion des tableaux triés

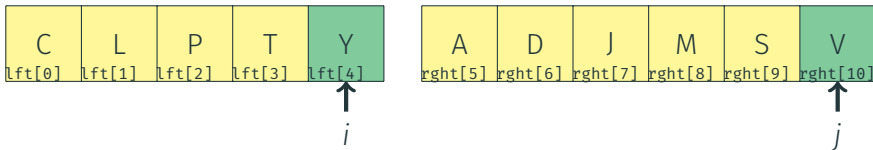


Tri fusion : illustration

Tableau initial :



Combiner : fusion des tableaux triés



Tri fusion : illustration

Tableau initial :

A	C	D	J	L	M	P	S	T	V	Y
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

↑
k

Combiner : fusion des tableaux triés

C	L	P	T	Y
lft[0]	lft[1]	lft[2]	lft[3]	lft[4]

A	D	J	M	S	V
rght[5]	rght[6]	rght[7]	rght[8]	rght[9]	rght[10]

Tri fusion : implémentation

```
def merge_sort(T):
    if len(T) >= 2:
        m = len(T)//2
        left = T[:m]
        merge_sort(left)
        right = T[m:]
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                T[k] = left[i]
                k, i = k+1, i+1
            else:
                T[k] = right[j]
                k, j = k+1, j+1
        while i < len(left):
            T[k] = left[i]
            k, i = k+1, i+1
        while j < len(right):
            T[k] = right[j]
            k, j = k+1, j+1
```

Tri fusion : implémentation

```
def merge_sort(T):
    if len(T) >= 2:
        m = len(T)//2
        left = T[:m]
        merge_sort(left)
        right = T[m:]
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                T[k] = left[i]
                k, i = k+1, i+1
            else:
                T[k] = right[j]
                k, j = k+1, j+1
        while i < len(left):
            T[k] = left[i]
            k, i = k+1, i+1
        while j < len(right):
            T[k] = right[j]
            k, j = k+1, j+1
```

Analyse :

Coût hors appel récursif : $O(n)$.

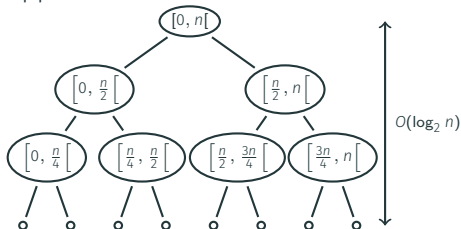
Tri fusion : implémentation

```
def merge_sort(T):  
    if len(T) >= 2:  
        m = len(T)//2  
        left = T[:m]  
        merge_sort(left)  
        right = T[m:]  
        merge_sort(right)  
        i = j = k = 0  
        while i < len(left) and j < len(right):  
            if left[i] <= right[j]:  
                T[k] = left[i]  
                k, i = k+1, i+1  
            else:  
                T[k] = right[j]  
                k, j = k+1, j+1  
        while i < len(left):  
            T[k] = left[i]  
            k, i = k+1, i+1  
        while j < len(right):  
            T[k] = right[j]  
            k, j = k+1, j+1
```

Analyse :

Coût hors appel récursif : $O(n)$.

Appels récursifs :



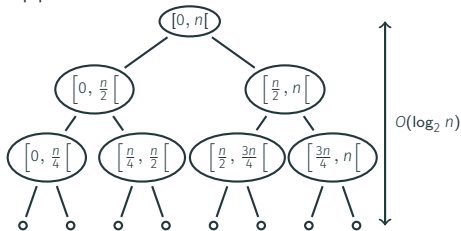
Tri fusion : implémentation

```
def merge_sort(T):  
    if len(T) >= 2:  
        m = len(T)//2  
        left = T[:m]  
        merge_sort(left)  
        right = T[m:]  
        merge_sort(right)  
        i = j = k = 0  
        while i < len(left) and j < len(right):  
            if left[i] <= right[j]:  
                T[k] = left[i]  
                k, i = k+1, i+1  
            else:  
                T[k] = right[j]  
                k, j = k+1, j+1  
        while i < len(left):  
            T[k] = left[i]  
            k, i = k+1, i+1  
        while j < len(right):  
            T[k] = right[j]  
            k, j = k+1, j+1
```

Analyse :

Coût hors appel récursif : $O(n)$.

Appels récursifs :



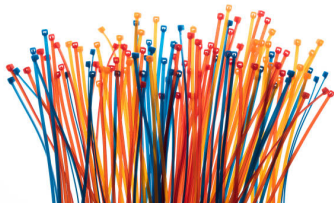
Complexité : $O(n \log_2 n)$.

$$n + \underbrace{\frac{n}{2} + \frac{n}{2}}_{2 \text{ fois}} + \underbrace{\frac{n}{4} + \dots + \frac{n}{4}}_{4 \text{ fois}} + \dots$$

Définition

Une stratégie par programmation dynamique exploite

- la propriété de sous-structure optimale
- le fait que les sous-instances se chevauchent.



Deux approches de la programmation dynamique

- Descendante**
- Écrire les **relations de récurrence** sous forme d'équations mathématiques .
 - En déduire un programme **récuratif**.
 - **Mémoïser** pour éviter l'explosion des appels.

Deux approches de la programmation dynamique

Descendante

- Écrire les **relations de récurrence** sous forme d'équations mathématiques .
- En déduire un programme **récuratif**.
- **Mémoïser** pour éviter l'explosion des appels.

Ascendante

- Anticiper les relations de **précédence** entre instances.
- **Remplir un tableau** de solutions dans un ordre compatible.

Deux approches de la programmation dynamique

Descendante

- Écrire les **relations de récurrence** sous forme d'équations mathématiques .
- En déduire un programme **récuratif**.
- **Mémoïser** pour éviter l'explosion des appels.

Ascendante

- Anticiper les relations de **précédence** entre instances.
- **Remplir un tableau** de solutions dans un ordre compatible.

Pour les puristes, seule la seconde approche est la « vraie » programmation dynamique.

Le rendu du monnaie : relations de récurrence

Entrées : Entier $m \in \mathbb{N}$ (le montant à rendre),
tuple $(s_1, \dots, s_k) \subseteq \mathbb{N}^k$ (les pièces possibles).

Préconditions : $s_1 > s_2 > \dots > s_k > 0$

Valeur de retour : Tuple $(a_1, \dots, a_k) \in \mathbb{N}^k$

Postconditions : $m = a_1 s_1 + \dots + a_k s_k$
 $\rho(m) = \sum_{i=1}^k a_i$ est aussi petit que possible.

Le rendu du monnaie : relations de récurrence

Entrées : Entier $m \in \mathbb{N}$ (le montant à rendre),
tuple $(s_1, \dots, s_k) \subseteq \mathbb{N}^k$ (les pièces possibles).

Préconditions : $s_1 > s_2 > \dots > s_k > 0$

Valeur de retour : Tuple $(a_1, \dots, a_k) \in \mathbb{N}^k$

Postconditions : $m = a_1 s_1 + \dots + a_k s_k$
 $\rho(m) = \sum_{i=1}^k a_i$ est aussi petit que possible.

Théorème

$$\rho(m) = \begin{cases} 0 & \text{si } m = 0 \end{cases}$$

Le rendu du monnaie : relations de récurrence

Entrées : Entier $m \in \mathbb{N}$ (le montant à rendre),
tuple $(s_1, \dots, s_k) \subseteq \mathbb{N}^k$ (les pièces possibles).

Préconditions : $s_1 > s_2 > \dots > s_k > 0$

Valeur de retour : Tuple $(a_1, \dots, a_k) \in \mathbb{N}^k$

Postconditions : $m = a_1 s_1 + \dots + a_k s_k$
 $\rho(m) = \sum_{i=1}^k a_k$ est aussi petit que possible.

Théorème

$$\rho(m) = \begin{cases} 0 & \text{si } m = 0 \\ 1 + \min_{i \in \llbracket 1, k \rrbracket} \rho(m - s_i) & \text{s'il existe } i \in \llbracket 1, k \rrbracket \text{ tel que } m \geq s_i \text{ et } \rho(m - s_i) \neq \infty \end{cases}$$

Le rendu du monnaie : relations de récurrence

Entrées : Entier $m \in \mathbb{N}$ (le montant à rendre),
tuple $(s_1, \dots, s_k) \subseteq \mathbb{N}^k$ (les pièces possibles).

Préconditions : $s_1 > s_2 > \dots > s_k > 0$

Valeur de retour : Tuple $(a_1, \dots, a_k) \in \mathbb{N}^k$

Postconditions : $m = a_1 s_1 + \dots + a_k s_k$
 $\rho(m) = \sum_{i=1}^k a_k$ est aussi petit que possible.

Théorème

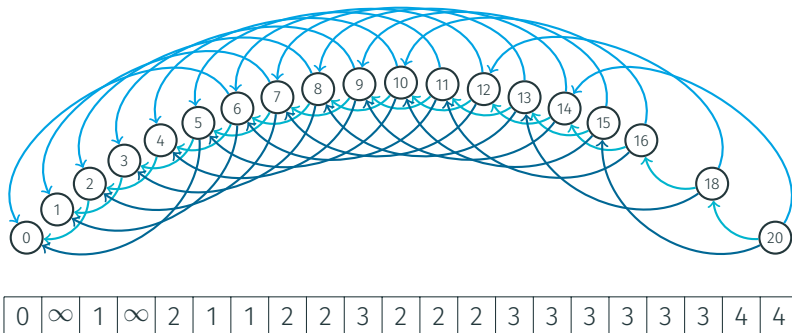
$$\rho(m) = \begin{cases} 0 & \text{si } m = 0 \\ 1 + \min_{i \in \llbracket 1, k \rrbracket} \rho(m - s_i) & \text{s'il existe } i \in \llbracket 1, k \rrbracket \text{ tel que } m \geq s_i \text{ et } \rho(m - s_i) \neq \infty \\ \infty & \text{sinon} \end{cases}$$

Le rendu du monnaie : approche descendante

```
def coin_change_topdown_dp(m, S, memo = None):
    if memo == None:
        memo = dict({0:0})
    if m not in memo:
        # we look for the recursive calls
        rhos = [coin_change_topdown_dp(m-s, S, memo) for s in
            ↪ S if m-s >= 0]
        # we filter out the infinity values
        rhos = [x for x in rhos if x is not None]
        if not rhos: # rhos is the empty list
            memo[m] = None
        else: # rhos is not the empty list
            # the recurrence equation applies
            memo[m] = 1 + min(rhos)
    return memo[m]
```

Graphe des appels récursifs

Avec $S = \{2, 5, 6\}$, sur $m = 20$.



Le rendu du monnaie : approche ascendante

```
def coin_change_bottomup_dp(m, S):  
    memo = [None for _ in range(m+1)]  
    memo[0] = 0  
    for i in range(1,m+1):  
        rhos = [memo[i-s] for s in S if i-s>=0]  
        rhos = [x for x in rhos if x is not None]  
        if not rhos: # rhos is the empty list  
            memo[i] = None  
        else: # rhos is not the empty list  
            memo[i] = 1 + min(rhos)  
    return memo[m]
```

Recourir à l'aléa

Algorithmes probabilistes

Définition

Un **algorithme probabiliste** est un algorithme qui a la possibilité de recourir à des tirages aléatoires et uniformes d'un bit.

Définition

Un algorithme **déterministe** désigne un algorithme qui produit toujours la même sortie en passant par les mêmes états.



Intérêt des algorithmes probabilistes

- **plus simples** à écrire que des algorithmes déterministes de complexité équivalente.
- **plus rapides** ou déjouent des complexités en pires cas sur des données structurées.





Définition

Un algorithme de type **Las Vegas** produit toujours un résultat systématiquement correct mais son temps d'exécution peut varier selon les tirages effectués.

Catégories d'algorithmes probabilistes



Définition

Un algorithme de type **Las Vegas** produit toujours un résultat systématiquement correct mais son temps d'exécution peut varier selon les tirages effectués.

Définition

Un algorithme de type **Monte Carlo** produit un résultat parfois incorrect selon les tirages effectués mais son temps d'exécution ne varie pas.



Problème des n dames : description

Entrée : Entier $n \in \mathbb{N}$ (taille de l'échiquier).

Précondition : $n > 0$

Valeur de retour : Tuple $(r_1, \dots, r_n) \in \llbracket 0, n-1 \rrbracket^n$

Postcondition : Dans le placement de n dames aux positions $(c, r_c)_{c \in \llbracket 0, n-1 \rrbracket}$, deux dames ne se menacent pas.

Exemple

	Col 0	Col 1	Col 2	Col 3
Lig 3				
Lig 2				
Lig 1				
Lig 0				

$[1, 3, 0, 2]$

	Col 0	Col 1	Col 2	Col 3
Lig 3				
Lig 2				
Lig 1				
Lig 0				

$[1, 0, 3, 2]$

Problème des n dames : principe général

Algorithme 1 : Recherche randomisée

```
1 répéter
2   pour  $c \in \llbracket 0, n - 1 \rrbracket$  faire
3     Calculer l'ensemble  $A$  des positions encore
       possibles.
4     si  $A$  est vide alors
5       Interrompre la recherche
6     sinon
7       Choisir une ligne  $r_c$  aléatoirement parmi
         les positions encore possibles.
8 jusqu'à une solution a été trouvée;
```

Problème des n dames : bilan

- L'algorithme de recherche randomisée est de type Las Vegas.
- Il est bien différent d'un algorithme par retour sur trace : en cas d'échec, la solution est abandonnée.
- En pratique, il fonctionne plus rapidement que la recherche par retour sur trace.

~1959



Sir [Tony] Hoare
(1934)

Principe

Diviser Piocher une valeur pivot p parmi celle de T
Réorganiser le tableau avec l'algorithme du **drapeau hollandais** :

- Rouge : $T[x]$ avec $T[x] < p$
- Bleu : $T[x]$ avec $T[x] > p$

Régner Trier la partie rouge et la partie bleue.

Combiner Rien à faire!

Tri rapide : illustration

Tableau initial :

L	C	Y	P	T	D	V	A	M	J	S
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Tri rapide : illustration

Tableau initial :

L	C	Y	P	T	D	V	A	M	J	S
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Diviser On choisit un pivot : ici P.

Tri rapide : illustration

Tableau initial :

L	C	J	M	D	A	P	V	T	S	Y
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Diviser On réorganise le tableau en temps $O(n)$ et en place.

Tri rapide : illustration

Tableau initial :

A	C	D	J	L	M	P	V	T	S	Y
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Régner On trie récursivement la partie gauche

Tri rapide : illustration

Tableau initial :

A	C	D	J	L	M	P	S	T	V	Y
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Régner On trie récursivement la partie droite.

Tri rapide : illustration

Tableau initial :

A	C	D	J	L	M	P	S	T	V	Y
T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]	T[8]	T[9]	T[10]

Combiner Il n'y a rien à faire dans ce cas.

Théorème

La complexité en moyenne du tri rapide est $O(n \log_2 n)$.

Le tri rapide : complexité

Théorème

La complexité en moyenne du tri rapide est $O(n \log_2 n)$.

Démonstration.

[Esquisse] Notons $\sigma_0, \dots, \sigma_{n-1}$ les éléments ordonnés.

Deux éléments σ_i et σ_j sont comparés seulement si l'un des deux est le premier élément parmi $\sigma_i, \sigma_{i+1}, \dots, \sigma_j$ à être choisi comme pivot, ce qui se produit avec probabilité $\frac{2}{|j-i|+1}$.

Ainsi il y a en moyenne $\sum_{i < j} \frac{2}{j-i+1}$ comparaisons. Or

$$\sum_{i < j} \frac{2}{j-i+1} = \Theta(n \log_2 n).$$



- L'algorithme de tri rapide randomisée est de type Las Vegas.

Le tri rapide : bilan

- L'algorithme de tri rapide randomisée est de type Las Vegas.
- Il déjoue le pire des cas d'un tri rapide déterministe où le pivot est le 1er élément.
Pire des cas : liste déjà triée.

Le tri rapide : bilan

- L'algorithme de tri rapide randomisée est de type Las Vegas.
- Il déjoue le pire des cas d'un tri rapide déterministe où le pivot est le 1er élément.
Pire des cas : liste déjà triée.
- C'est un algorithme de tri en place. Le coût en mémoire supplémentaire est $O(1)$.

Test de Fermat : principe

XVII^e s.



Pierre de Fermat

Théorème (Fermat)

Si p est un nombre premier et si a est un entier non divisible par p , alors $a^{p-1} \equiv 1 \pmod{p}$.

Test de Fermat : principe

XVII^e s.



Pierre de Fermat

Théorème (Fermat)

Si p est un nombre premier et si a est un entier non divisible par p , alors $a^{p-1} \equiv 1 \pmod{p}$.

Principe du test

- Tirer au sort $a \in \llbracket 2, p-1 \rrbracket$.

Test de Fermat : principe

XVII^e s.



Pierre de Fermat

Théorème (Fermat)

Si p est un nombre premier et si a est un entier non divisible par p , alors $a^{p-1} \equiv 1 \pmod{p}$.

Principe du test

- Tirer au sort $a \in \llbracket 2, p-1 \rrbracket$.
- Calculer $x = a^{p-1} - 1 \pmod{p}$.

Test de Fermat : principe

XVII^e s.



Pierre de Fermat

Théorème (Fermat)

Si p est un nombre premier et si a est un entier non divisible par p , alors $a^{p-1} \equiv 1 \pmod{p}$.

Principe du test

- Tirer au sort $a \in \llbracket 2, p-1 \rrbracket$.
- Calculer $x = a^{p-1} - 1 \pmod{p}$.
- Si x est non nul, on a la certitude que p n'est pas premier.
Si x est nul, on a la présomption que p est premier (mais on peut se tromper!).

- Le test de Fermat est un algorithme randomisé de type Monte Carlo.

- Le test de Fermat est un algorithme randomisé de type **Monte Carlo**.
- La probabilité que le test se trompe est élevé, mais une variante (le test de **Miller-Rabin**) fait mieux avec les mêmes idées.

- Le test de Fermat est un algorithme randomisé de type **Monte Carlo**.
- La probabilité que le test se trompe est élevé, mais une variante (le test de **Miller-Rabin**) fait mieux avec les mêmes idées.
- Il est emblématique des tests de primalité utilisés industriellement en **cryptographie** : il est extrêmement rapide et la probabilité de rendre une réponse fausse peut être rendue tellement petite que l'on peut s'y fier.

Algorithme de Freivalds : objectifs

Entrées : Triplet de matrices $\mathbf{A} \in \mathbb{K}^{n \times n}$, $\mathbf{B} \in \mathbb{K}^{n \times n}$ et $\mathbf{C} \in \mathbb{K}^{n \times n}$.

Valeur de retour : Booléen **vrai** si $\mathbf{AB} = \mathbf{C}$ et **faux** sinon.

Algorithme de Freivalds : objectifs

Entrées : Triplet de matrices $\mathbf{A} \in \mathbb{K}^{n \times n}$, $\mathbf{B} \in \mathbb{K}^{n \times n}$ et $\mathbf{C} \in \mathbb{K}^{n \times n}$.

Valeur de retour : Booléen **vrai** si $\mathbf{AB} = \mathbf{C}$ et **faux** sinon.

Complexité de l'approche naïve

La multiplication naïve de matrice est en $O(n^3)$, donc tester $\mathbf{AB} = \mathbf{C}$ se fait en $O(n^3)$.

Par des algorithmes avancés, la multiplication de matrice est en $O(n^\omega)$ où $\omega \simeq 2,372$.

1977

Principe de l'algorithme

- Tirer au sort un vecteur $\mathbf{x} \in \{0, 1\}^n$.

- Calculer $\mathbf{y} = \mathbf{A}(\mathbf{B}\mathbf{x})$ et $\mathbf{z} = \mathbf{C}\mathbf{x}$.

- Si $\mathbf{y} \neq \mathbf{z}$, on a la certitude que $\mathbf{AB} \neq \mathbf{C}$.

Si $\mathbf{y} = \mathbf{z}$, on a la présomption que $\mathbf{AB} = \mathbf{C}$ (mais on peut se tromper!).



Rūsiņš Mārtiņš Freivalds
(1948-2016)

1977



Rūsiņš Mārtiņš Freivalds
(1948-2016)

Principe de l'algorithme

- Tirer au sort un vecteur $\mathbf{x} \in \{0, 1\}^n$.
- Calculer $\mathbf{y} = \mathbf{A}(\mathbf{B}\mathbf{x})$ et $\mathbf{z} = \mathbf{C}\mathbf{x}$.
- Si $\mathbf{y} \neq \mathbf{z}$, on a la certitude que $\mathbf{AB} \neq \mathbf{C}$.
Si $\mathbf{y} = \mathbf{z}$, on a la présomption que $\mathbf{AB} = \mathbf{C}$ (mais on peut se tromper!).

Optimalité de la complexité de l'algorithme de Freivalds

Complexité en $O(n^2)$, i.e. **linéaire** en la taille des entrées.

Or, tout algorithme est $\Omega(n^2)$ puisqu'il faut au moins lire les coefficients des trois matrices.

Algorithme de Freivalds : analyse

Théorème

Si $\mathbf{AB} = \mathbf{C}$, l'algorithme ne se trompe jamais. Si $\mathbf{AB} \neq \mathbf{C}$, alors l'algorithme de Freivalds se trompe avec probabilité $\leq \frac{1}{2}$.

Algorithme de Freivalds : analyse

Théorème

Si $\mathbf{AB} = \mathbf{C}$, l'algorithme ne se trompe jamais. Si $\mathbf{AB} \neq \mathbf{C}$, alors l'algorithme de Freivalds se trompe avec probabilité $\leq \frac{1}{2}$.

Démonstration.

Si $\mathbf{AB} = \mathbf{C}$, il est clair que $\mathbf{ABx} = \mathbf{Cx}$.

Si $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}_{n \times n}$, il existe un coefficient de $\mathbf{D}$ non nul, qu'on peut supposer, sans perte de généralité, en première ligne et colonne.

Algorithme de Freivalds : analyse

Théorème

Si $\mathbf{AB} = \mathbf{C}$, l'algorithme ne se trompe jamais. Si $\mathbf{AB} \neq \mathbf{C}$, alors l'algorithme de Freivalds se trompe avec probabilité $\leq \frac{1}{2}$.

Démonstration.

Si $\mathbf{AB} = \mathbf{C}$, il est clair que $\mathbf{ABx} = \mathbf{Cx}$.

Si $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}_{n \times n}$, il existe un coefficient de $\mathbf{D}$ non nul, qu'on peut supposer, sans perte de généralité, en première ligne et colonne.

Quand $x_2, x_3, \dots, x_n$ sont fixés, avec x_1 tiré dans $\{0, 1\}$,

$$\mathbb{P}(d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n = 0) \leq \frac{1}{2}$$

Algorithme de Freivalds : analyse

Théorème

Si $\mathbf{AB} = \mathbf{C}$, l'algorithme ne se trompe jamais. Si $\mathbf{AB} \neq \mathbf{C}$, alors l'algorithme de Freivalds se trompe avec probabilité $\leq \frac{1}{2}$.

Démonstration.

Si $\mathbf{AB} = \mathbf{C}$, il est clair que $\mathbf{ABx} = \mathbf{Cx}$.

Si $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}_{n \times n}$, il existe un coefficient de $\mathbf{D}$ non nul, qu'on peut supposer, sans perte de généralité, en première ligne et colonne.

Quand $x_2, x_3, \dots, x_n$ sont fixés, avec x_1 tiré dans $\{0, 1\}$,

$$\mathbb{P}(d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n = 0) \leq \frac{1}{2}$$

Mais

$$\begin{aligned} \mathbb{P}(\mathbf{Dx} = \mathbf{0}) &\leq \mathbb{P}(d_{1,1}x_1 + d_{1,2}x_2 + \dots + d_{1,n}x_n = 0) \\ &= \frac{1}{2^{n-1}} \sum_{(\hat{x}_i)_{2 \leq i \leq n} \in \{0,1\}^{n-1}} \mathbb{P}(d_{1,1}x_1 + d_{1,2}\hat{x}_2 + \dots + d_{1,n}\hat{x}_n = 0) \\ &\leq \frac{1}{2} \end{aligned}$$